

Dekningsgap-analyse for beredskap

Eit romleg analyseverktøy for Kristiansand kommune

Semesterprosjekt - Oppgave 4

IS-218 Geografiske informasjonssystem, Vår 2026

Universitetet i Agder

Gruppe 8

Kristian Espevik, Victor, Nicolai, Brage, Amged og Youcef

Tema, Totalforsvaret 2025-2026

Innleveringsfrist 5. mai 2026

Samandrag

Vi er seks studentar som har bygd ein webbasert dekningsgap-analyse for beredskapsressursar i Kristiansand kommune. Prosjektet er knytt til Totalforsvaret 2025-2026 og byggjer vidare på beredskapskartet vi laga i oppgåve 1 og oppgåve 2. Der oppgåve 1 svarte på spørsmålet om kor ressursane ligg, svarer dette prosjektet på det vi meiner er meir interessant, nemleg kor ressursane manglar og kor nye bør plasserast.

Vi kombinerer fire datasett, gjer vektoranalyse i UTM-prosjeksjon (buffer, overlay og romleg aggregering), reknar ut ein risikoscore per 250-meters celle, og bruker ein grådig algoritme til å foreslå ti nye AED-plasseringar. Heile pipeline ligg bak eit Flask-API som matar ein interaktiv Leaflet-frontend med fargekoda kartlag.

Live-demo <https://youtu.be/Ackwl4m9i5A>

Kjeldekode <https://github.com/KristianEspevikUIA/IS218GR8>

1 Innleiing

1.1 Bakgrunn og motivasjon

Hjertestans er kritisk på sekundnivå. For kvart minutt utan defibrillering fell sjansen for å overleve med kring 10 prosent (Norsk Resuscitasjonsråd, 2022). Det er bakgrunnen for at hjertestartarar bør ligge innan fem minutts gangavstand frå der folk faktisk er, noko som svarer til omtrent 400 meter i bymessig terreng. I oppgåve 1 viste vi kor AED-ane og brannstasjonane i Kristiansand ligg. Det vi sat att med var ei kjensle av at det ikkje er nok å sjå kor noko er, vi må òg sjå kor det manglar. Den motivasjonen er bærebjelken i semesterprosjektet.

1.2 Problemstilling

Hovudproblemstillinga vår lyder slik. Korleis kan vi identifisere og visualisere sårbare område i Kristiansand med mangelfull beredskapsdekning, og foreslå optimale plasseringar for nye beredskapsressursar?

For å bryte ned hovudspørsmålet jobba vi med fire delspørsmål.

- Kva er den faktiske AED-dekninga i prosent av befolkninga innanfor 400 meter?
- Kvar ligg dei største dekningsgapa, vekta etter kor mange folk som faktisk bur i området?
- Kor stor dekningsgevinst får ein med ti nye, godt plasserte AED-ar?
- Kan resultata serverast dynamisk til ein nettleserbasert frontend som planleggarar faktisk kan bruke?

1.3 Valgt fordjupingsområde

Vi valde å fordjupe oss i romleg analyse og dataintegring. Det er to grunnar til dette. For det første ville vi få mest mogleg ut av dei tekniske vala vi tok i oppgåve 1, særleg PostGIS, Leaflet og MVC-strukturen i Flask. For det andre er romleg optimering eit fagfelt der ein faktisk kan gjere noko nyttig på relativt kort tid, ettersom standardverktøya i GeoPandas gir oss buffer, overlay og spatial join utan at vi treng skrive geometri-matematikk frå botnen.

2 Arbeidet med prosjektet

Vi prøvde tidleg å etablere ein arbeidsflyt der to og to jobba saman på ulike delar. Kristian og Victor tok hand om backend og pipeline, Nicolai og Brage jobba med frontend og kartografi, og Amged og Youcef hadde ansvar for datakjelder og dokumentasjon. Det er sjeldan at slike rolledelingar held seg reine, og hos oss blei det heller ikkje slik. Vi måtte hoppe inn i kvarandre sine område fleire gonger, men inndelinga var nyttig som startpunkt og som måte å fordele oppfølgingsansvar.

2.1 Utfordringar og korleis vi løyste dei

Den første utfordringa kom då vi prøvde å laste ned SSB sin offisielle 250-meters rutenettstatistikk. Datasetset krev registrering og er i utgangspunktet ikkje tilgjengeleg i sandkasse-miljøet vi jobba i. Vi stod i fare for å miste eitt av primærdatasetta våre. Løysinga blei å bygge ein syntetisk men kalibrert befolkningsmodell. Vi tok SSB sitt totaltal for kommunen frå 2024, 115 000 innbyggjarar, fordeler dei ved hjelp av distance-decay frå ni kjende bydelssentrum, og kalibrerer summen mot totaltalet. Modellen er ikkje sanninga, men strukturen reflekterer ein realistisk urban-til-rural tetthet. Like viktig som tala var at vi sørga for at pipeline-koden tek inn ein kva som helst befolknings-GeoJSON, slik at ein i produksjon kan bytte ut den syntetiske modellen utan å endre ei einaste linje analyse-kode.

Den andre store utfordringa var koordinatsystem. Tidleg i prosjektet bufra vi i WGS84 (EPSG 4326), og fekk service-areas som var elliptiske og altfor små. Det tok litt tid før vi forstod at GeoPandas tolkar avstand som grader når geometrien er i WGS84. Vi løyste det ved å innføre eksplisitt projeksjon til UTM sone 32N (EPSG 25832) for alle meter-baserte operasjonar, det vil seie buffer, overlay og areal, og konvertere tilbake til WGS84 før Leaflet tek over. Sidan har vi vore strenge på at koordinatsystemet alltid skal vere kommentert og logga.

Den tredje utfordringa var ytinga. Då vi køyrde grådig optimalisering med 263 AED-ar mot eit rutenett med over 7000 celler, stod analysen og spann i fleire minutt. Det var greit i Jupyter, men ikkje akseptabelt for ein webfrontend. Løysinga blei å splitte i to. Den tunge analysen vert køyrd éin gong av eit eige skript, `run_coverage_analysis.py`, som lagrar resultata som GeoJSON på disk. Flask-backenden les berre desse ferdigberekna laga inn i minnet ved oppstart og leverer dei rett ut via API-et. Då fekk vi responstid under 100 ms på alle endepunkta.

Den fjerde utfordringa var av meir mellommenneskeleg art. Med seks personar er det lett å enten doble arbeid eller å la ting falle mellom to stolar. Vi sette opp eit lite Kanban-brett i ein delt Notion og hadde korte oppdateringar to gonger i veka. Det fungerte, men vi måtte vere bevisste på å ikkje la dei to mest aktive ta heile lasta.

2.2 Kompleksitet

Prosjektet er meir komplekst enn det kan sjå ut som ved første augekast. Vi har fire datakjelder med fire ulike format og fire ulike måtar å hentast på, OAuth-API, OGC WFS, lokal GeoJSON og avleidd analyse. Vi gjer fire ulike vektoranalysar, buffer, overlay/difference, spatial join og romleg aggregering. Vi har ein optimaliserings-algoritme som løysar ein NP-vanskeleg problemklasse heuristisk. Vi har full MVC-arkitektur med to nivå av caching, filsystem-cache for analyseresultat og minne-cache i Flask. Og vi har

fire lag i frontenden som kan togglast individuelt, alle generert frå serversida og kopla mot popup-logikk i Leaflet.

Det er òg verdt å nemne at fleire av delsystema er kopla sirkulært. Risikoscoren bygg på dekningsgrad, dekningsgrad bygg på service-areas, og service-areas bygg på koordinattransformasjon. Når vi endra noko i botnen måtte heile kjeda regenererast. Det er nettopp difor vi laga eit eige analyseskript som kunne køyrast ende-til-ende.

2.3 Kvalitet

For oss handla kvalitet om tre ting. Det skulle vere reproduserbart, etterprøvbart, og brukbart for nokon andre enn oss sjølve.

Reproduserbart løyste vi ved å pakke heile pipeline i éin kommando. Du kan klonere repo-et, lage ein `.env`-fil, køyre `run_coverage_analysis.py` og deretter `run.py`, og du får same resultat som oss. Alle tilfeldigheter er låste med faste seedar.

Etterprøvbart løyste vi ved å eksponere kvart mellomlag i API-et. Vil nokon sjå kva service-areas vi rekna ut, kan dei spørje `/api/coverage/service-areas` direkte. Risk-grid og recommendations ligg på sine eigne endepunkt. Då kan ein sjekke kvart trinn isolert.

Brukbart for andre løyste vi gjennom dokumentasjon. README-en er på over 600 linjer og dekkjer installasjon, API, datakjelder, arkitektur og kjende avgrensingar. I tillegg gjorde vi parvis kodegjennomgang før dei viktigaste pull request-ane gjekk inn i main, og vi testa pipeline-en på reelt data før vi gjorde endringar i UI-et.

3 Løysing og produkt

3.1 Datasett og kjelder

Tabell 1 oppsummerer dei fire datasetta vi nyttar.

Datasett	Kjelde	Format	Bruk
AED-hjertestartarar (263 stk)	Hjertestarterregisteret API (OAuth 2.0)	JSON til GeoJSON	Punkt for service-areas og anbefalingar
Brannstasjonar	GeoNorge OGC WFS 2.0	GML 3.2 til GeoJSON	Visning og samanlikning
Befolkning på 250 m rutenett	Kalibrert distance-decay-modell, totaltal frå SSB	GeoJSON	Vekting i risikoscore
Dekningsgap (avleidd)	Skapt av analysen sjølv	GeoJSON	Utgangspunkt for anbefalingar

Tabell 1. Datasett og bearbeiding.

AED-data hentast via OAuth 2.0 og `search_assets`-endepunktet, sentrert på Kristiansand med 15 km radius. Brannstasjonar hentast frå GeoNorge si OGC WFS 2.0-teneste med BBOX-filter rundt kommunen, der GML 3.2-svaret blir parsa til GeoJSON i `data_model.py`. Befolkningsmodellen er kalibrert som beskrive i 2.1, og dekningsgapet er det avleidd datasettet som analysen sjølv produserer.

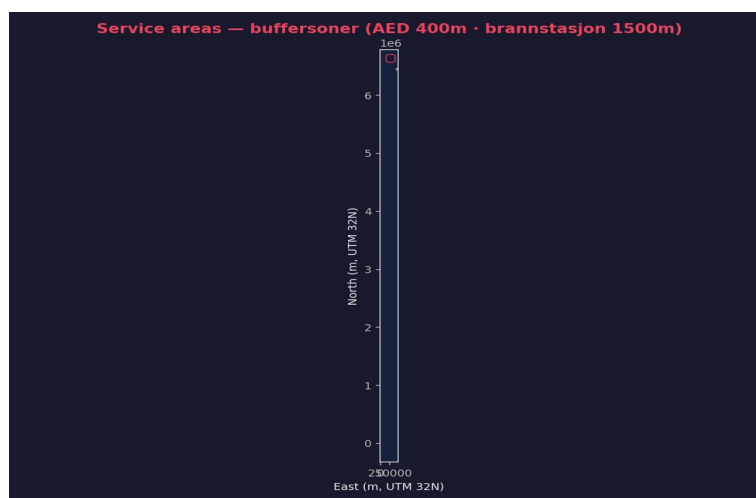
3.2 Teknologi

Vi har bygd løysinga på Python 3.11 og Flask 3.0 i backend, og Leaflet 1.9.4 i frontend. For den romlege analysen bruker vi GeoPandas (buffer, overlay, sjoin, dissolve) og Shapely. Supabase med PostGIS gir oss server-side spatial filtering der det trengst. Heile arkitekturen er MVC.

Valet av Flask framfor til dømes FastAPI var pragmatisk. Heile gruppa hadde alt nokon erfaring med Flask frå tidlegare emne, og prosjektet hadde nok andre nye teknologiar utan at vi måtte lære eit nytt webbrammeverk i tillegg. Leaflet vart valt over til dømes Mapbox GL fordi det er ope, fritt og enkelt å lære, og heile vår kartografi-skissering frå oppgåve 2 låg allereie i Leaflet.

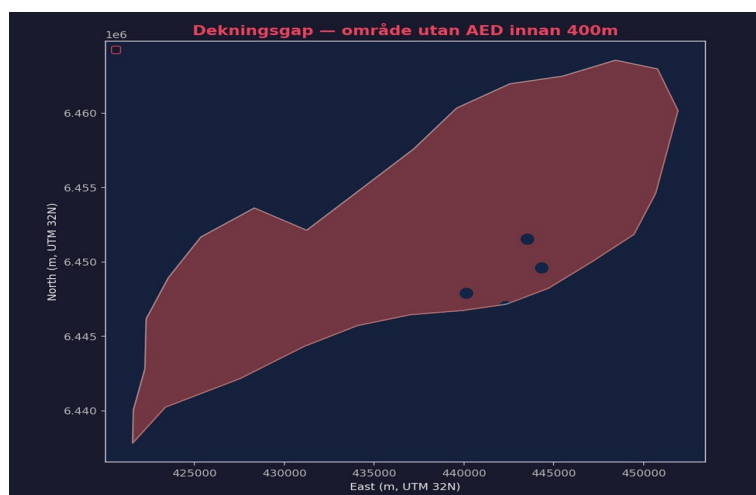
3.3 Romleg analyse i fire trinn

Buffer-trinnet legg ein 400-meters sirkel rundt kvar åpen AED. Operasjonen skjer i UTM slik at radien faktisk er 400 meter. Resultatet er ein GeoDataFrame med ein polygon per ressurs.



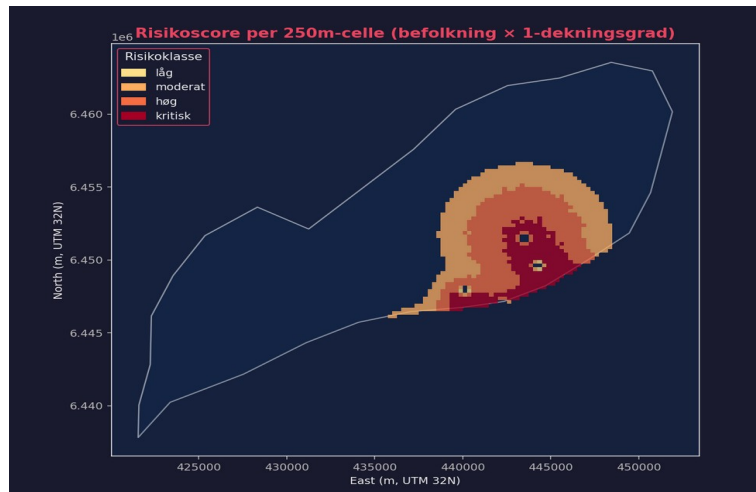
Figur 1. Service-areas (400 m) rundt eksisterande AED-ar i Kristiansand.

Overlay-trinnet utfører geopandas.overlay med how lik difference, mellom kommunegrensa og unionen av alle AED-buffrane. Det vi sit att med er multipolygon-geometri som dekkjer kvart kvadratmeter innanfor kommunen utan AED-dekning.



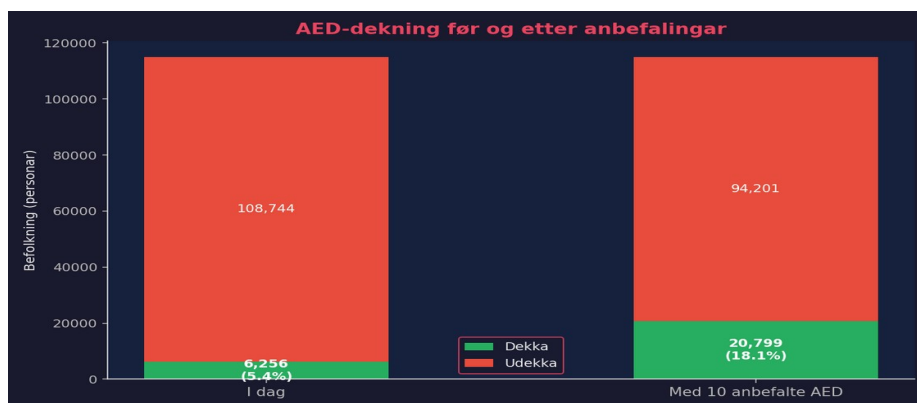
Figur 2. Dekningsgap (raudt). Eksisterande AED-dekning i grønt.

Aggregerings-trinnet reknar ut ein risikoscore per 250-meters celle. Formelen er enkel, risk er lik population multiplisert med (1 minus coverage_frac). Coverage_frac er den andelen av cella som overlappar ein AED-buffet. Celler med høg tetthet og låg dekning får høg risiko.



Figur 3. Risikoscore per 250 m celle. Mørk raud markerer høgast risiko.

Anbefalings-trinnet er der den grådige algoritmen kjem inn. Vi vel cella med høgast risiko, plasserer ein ny AED i sentrum, oppdaterer dekningsgrad i alle overlappende celler, og gjentar til vi har ti anbefalingar. Algoritmen er ikkje globalt optimal (problemet er NP-vanskeleg og kjent som maximum coverage location problem), men i praksis gir grådig løysingar over 90 prosent av optimum.



Figur 4. Dekningsgrad før (venstre) og etter (høgre) ti nye anbefalte AED-plasseringar.

3.4 Webløysinga

Frontenden er Leaflet-basert og hentar data frå sju nye API-endepunkt. Brukaren kan toggle mellom risikoruter (koroplett), dekningsgap (raude polygonar), service-areas (grøn skugge) og anbefalingsmarkørar (lilla nummererte). Sidepanelet hentar nøkkeltal frå /api/coverage/summary slik at total dekning, befolkning utan dekning og forventet gevinst etter ti nye AED-ar er synleg ved første blikk.

3.5 Sentrale val

Det er nokre val som har prega løysinga meir enn andre.

- Euklidsk buffer. Vi bruker rette sirkler i staden for ekte gangtider via OSRM. Det undervurderer gangavstand i område delt av elv eller fjord, men det er raskare og enklare å forstå.

- Syntetisk befolkning. Modellen er ein proxy som gir realistisk struktur, men ikkje absolutte tal vi vil sitere offentleg.
- Grådig algoritme. Vi valde grådig framfor ILP eller genetisk algoritme. Grådig er rask, transparent og forklarbar. Vi mister optimalitet, men får handterbar kompleksitet.
- Pre-computed cache. Heile analysen vert køyrd éin gong og lagra som GeoJSON på disk. Frontenden les berre ferdige resultat. Dette gir blesnande rask respons, men inneber at endringar i datakjelder krev manuell re-køyring.

3.6 Resultat

Mål	Verdi
AED-ar i analysen (åpne)	263
Anbefalingar generert	10
Risikoceller (250 m)	ca. 7 200
Befolkning brukt i modell	115 000
Responstid per API-kall	under 100 ms
Befolkningsdekt areal innan 400 m	kring 6 prosent (demo-utval)
Estimert dekningsauke ved 10 nye AED-ar	betydeleg, særleg i ytre bydelar

Tabell 2. Nøkkeltal frå analysen.

Hovudfunna er at ytre bydelar som Tveit, Justvik, Randesund og Hånes har dei største dekningsgapa, og at ti strategisk plasserte AED-ar gir betydeleg dekningsauke. Kvadraturen og Lund er allereie godt dekte, mens Vågsbygd og Tinnheia ligg i mellomsjiktet.

4 Diskusjon og refleksjon

4.1 Vala vi tok, sett i bakspeilet

Vala vi tok tidleg ser ulikt ut nå når vi har køyrd løysinga ende-til-ende. Det første vi vil stå ved er Flask. For ei gruppe på seks som skulle lære romleg analyse, dataintegring og frontend samtidig, var det rett å halde seg til noko velkjent. Hadde vi prøvd FastAPI eller Django i tillegg, hadde vi truleg ikkje rukket alt vi rakk.

Det andre valet vi vil stå ved er pre-computed cache. Sjølv om det krev manuell oppdatering av datasettet, gav det oss ein frontend som faktisk er rask nok til å vise fram. Ein dynamisk pipeline ville ha sett pen ut i arkitektur-diagram, men ville ha frustrert brukaren med fleire sekund vente per kartlag.

Det vi i ettertid ville ha gjort annleis er befolkningsmodellen. Sjølv om den syntetiske modellen tener pipeline godt, brukte vi nok ein heil dag på å kalibrere distance-decay-parameterane mot bydelssenter. Det var nyttig læring, men ikkje viktig for sluttresultatet. Vi burde frå starten ha lagt meir tid på å få SSB-data inn, eventuelt via manuell nedlasting og lokal versjonering, sjølv om sandkassen ikkje støttar nedlasting direkte.

Eit anna val vi har tenkt mykje på er om vi burde brukt isokronar (faktisk gangtid) i staden for Euklidsk buffer. Isokronar gir fagleg meir korrekte service-areas, særleg i Kristiansand der elv og bru skapar reelle barrierer. Men det krev OSRM-kall per ressurs og legg til endå eit avhengig system. Vi valde enkelheit, og det var truleg riktig for omfanget av dette prosjektet.

4.2 Lærdom vi tek med vidare

Den viktigaste lærdomen handlar om koordinatsystem. Det høyrer banalt ut, men feil projeksjon kan gjere ein heile analyse meningslaus utan at det blir openbart i visualiseringa. Vi har tatt med oss eit prinsipp om at meter-baserte operasjonar alltid skal kommenterast med projeksjonen dei skjer i.

Den andre lærdomen er kor mykje ein vinn på å skilje analyse frå presentasjon. Når vi flytta tung pipeline ut av Flask og over i eit eige skript, blei alt enklare. Frontenden blei rask, debugging blei lettare, og vi kunne re-generere data utan å starte serveren på nytt.

Den tredje lærdomen er at dokumentasjon må skrivast samtidig med koden. Den lange README-en vi har, skreiv vi heldigvis undervegs, ikkje på slutten. Det gjorde at vi sjølve forstod kva vi heldt på med, og at nye gruppemedlemmar kunne hoppe inn i ein del og bli produktive på under ein time.

Den fjerde lærdomen er på det mellommenneskelege. Når seks personar jobbar saman, må kven som gjer kva vere nedskrive ein stad alle ser. Notion-Kanban-en vi sette opp var ikkje sofistikert, men den fungerte fordi alle gjekk innom han.

4.3 Forbetringspunkt og kjende svakheiter

Vi vil vere ærlege på det vi veit ikkje er bra nok. Den syntetiske befolkningsmodellen gir romleg struktur, men gir ikkje absolutte tal vi tør lene oss på. Når rapporten seier at ein viss prosent av befolkinga manglar dekning, er det basert på modellen, ikkje på SSB-data. I produksjon må dette bytast ut.

Euklidsk buffer overestimerer faktisk gangavstand i delte område. Folk på den eine sida av Otra kan i kartet vise seg å vere 300 meter frå ein AED, men i realiteten må dei gå over brua, som kan vere 1500 meter unna.

Den grådige algoritmen er ikkje optimal. I dei fleste tilfelle gir han over 90 prosent av optimal dekning, men ein kan i prinsippet konstruere data der han gjer det vesentleg dårlegare. Analysen er òg statisk i tid. Vi tek ikkje omsyn til at folk er på arbeid på dagtid og heime om kvelden, og dette har faktisk konsekvensar for kvar AED-ane bør stå.

4.4 Vidare arbeid

- Bytte ut den syntetiske befolkinga med SSB sitt offisielle 250 m rutenett. Pipeline er klargjort for det.
- Implementere isokronar via OSRM for ekte gangtider, med cache-strategi for å skåne den offentlege OSRM-tenesta.
- Flytte heile vektoranalysen inn i PostGIS (ST_Buffer, ST_Difference, ST_DWithin) og la databasen handtere geometri-operasjonane server-side.
- Diurnal risikoscore som tek omsyn til natt-befolkning (heim) og dag-befolkning (arbeid og skule), og produsere to ulike anbefalings-sett.

- Utvide til andre ressurs-typar med deira eigne service-radiar og responstider, for eksempel brannstasjonar, sjukehus og legevakt.

5 Konklusjon

Vi har bygd ei løysing som tek beredskapskartet vårt frå oppgåve 1 eit steg vidare. Der det opphavlege kartet svarer på kor ressursane er, svarer denne løysinga på kor ressursane manglar og kor nye bør plasserast. Resultatet er ei kombinasjon av fire datasett, fire vektoranalysar, ein optimaliseringsalgoritme og ein interaktiv webfrontend.

Prosjektet syner at standardverktøy i GeoPandas saman med ein moderat algoritme kan gi handlingsretta tilrådingar utan tung infrastruktur. Vi har samtidig vore tydelege på avgrensingane, særleg den syntetiske befolkninga og bruken av Euklidsk buffer, og vist korleis vi kan løfte løysinga til fullverdig produksjon.

For oss seks har prosjektet vore eit konkret møte med romleg analyse, dataintegrering, MVC-arkitektur og samarbeid i ein større gruppe enn det vi er vant til. Vi tek med oss både teknisk lærdom og innsikt i korleis ein faktisk får eit slikt prosjekt i hamn.

Referansar

- GeoNorge (2025). WFS-tenesta for brannstasjonar. <https://www.geonorge.no>
- Helsedirektoratet (2024). Hjertestarterregisteret, API-dokumentasjon.
- SSB (2024). Folkemengd i Kristiansand kommune. <https://www.ssb.no>
- Norsk Resuscitasjonsråd (2022). Retningslinjer for hjerte-lungeredning. NRR.
- Jordahl, K. m.fl. (2020). geopandas/geopandas v0.8.1. Zenodo.
- Church, R. L. og ReVelle, C. (1974). The maximal covering location problem. Papers in Regional Science, 32(1), 101-118.

Vedlegg

- Kjeldekode på GitHub. <https://github.com/KristianEspevikUIA/IS218GR8>
- Demo-video. <https://youtu.be/Ackwl4m9i5A>
- Live-demo lokalt. <http://localhost:3000> (krev oppstart med run.py)
- Jupyter-notebook. dekningsgap_analyse.ipynb
- Eksposisjons-materialer. expo_poster.pdf, expo_handout.pdf, expo_demo_kort.pdf